

How online games deal with lag

Anatoly Zavyalov
Boston University



Sniper's Nest



1:30
5 4



\$600



• [WERT-HUNTER] JOKA # Bomb site B • (RADIO): Smoke Out!
• [WERT-HUNTER] JOKA # Bomb site B • (RADIO): Fire in the hole!
• [WERT-HUNTER] JOKA # Bomb site B • (RADIO): Flashbang Out!
Affective is joining the Counter-Terrorist force
• Señor-Akro # Bomb site A • (RADIO): Flashbang Out!
◀ [WERT-HUNTER] Affective

100 0

10 / 30

The background is a dark blue field with a radial pattern of light blue lines. Scattered across the field are various pixelated characters and objects. In the top left, a red character with a Santa hat holds a hammer. In the top right, a purple character in a police uniform aims a submachine gun. On the left, a pink character with a yellow hard hat holds a revolver. In the center, a blue character with sunglasses holds a green bomb. Below that, a green character with a Viking helmet holds a sword. At the bottom left, a character in a green helmet is partially visible. On the right, a yellow character with a brown hat holds a handgun. A blue character with a red scarf and a winter hat is on the far right. Several grey square blocks are arranged in clusters. Three large, bright yellow circles with green outlines are positioned in the top left, center, and bottom right. The text 'SUPREM.IO' is centered in a bold, white, italicized font with a red drop shadow.

SUPREM.IO

Developing online games is demoralizing.

- When your game doesn't work, players get mad.
- When your game does work, players don't notice.
- Game networking is a black box to players.
- I will open up this box for you.

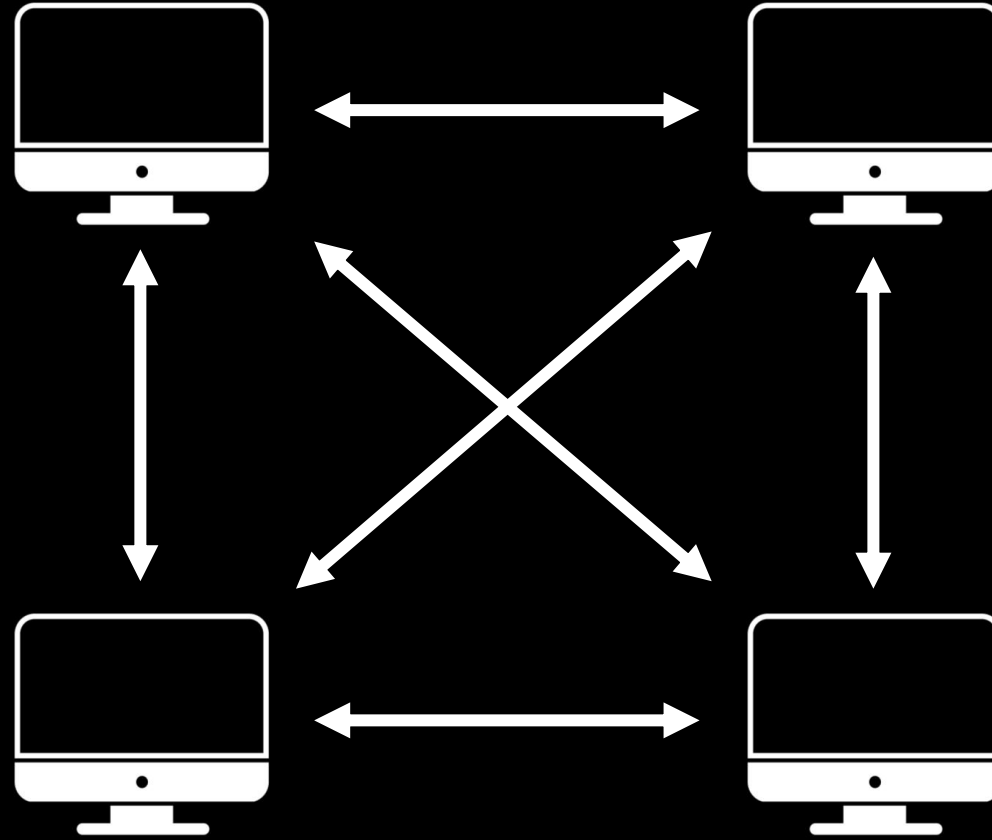


1999

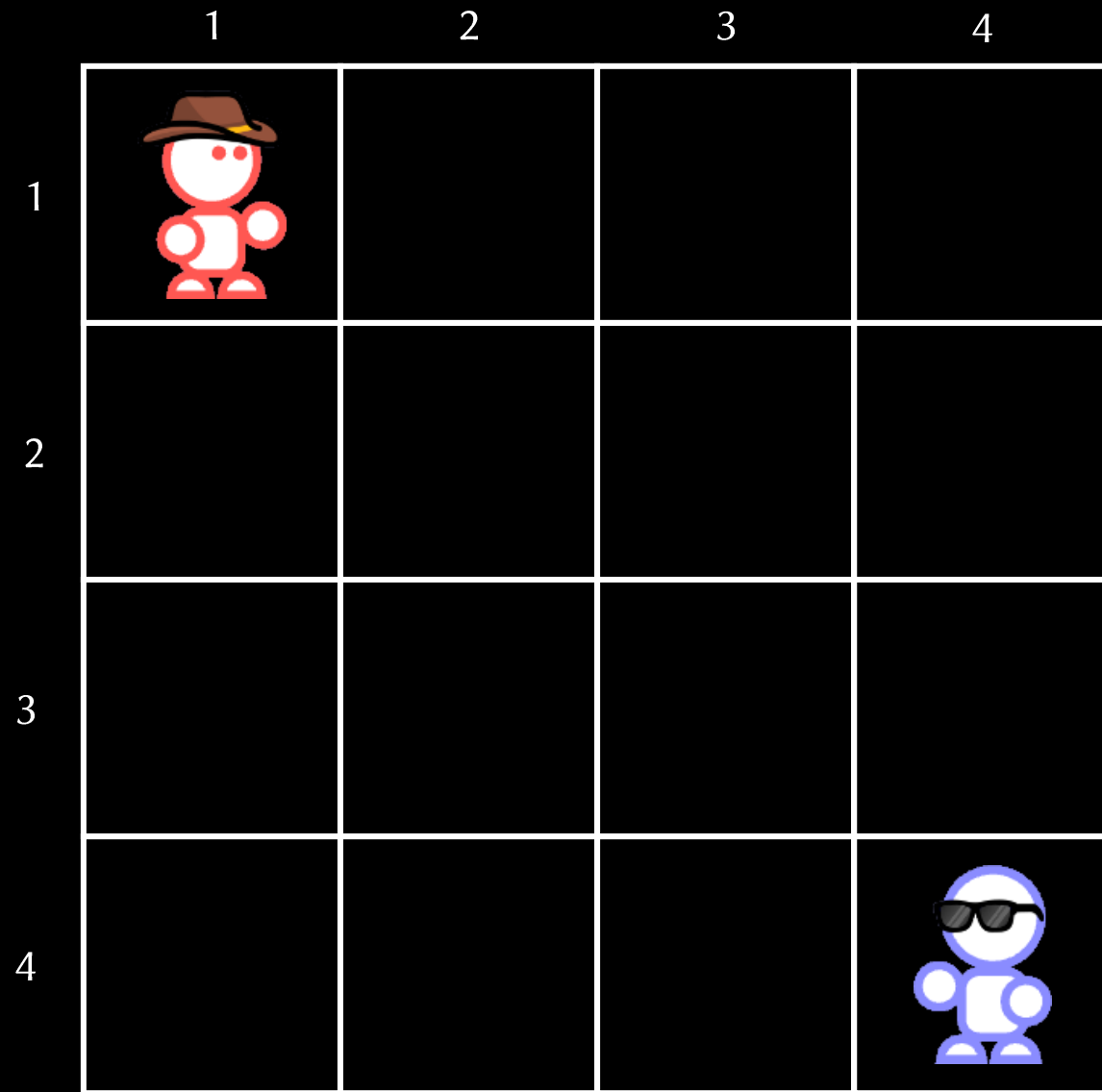
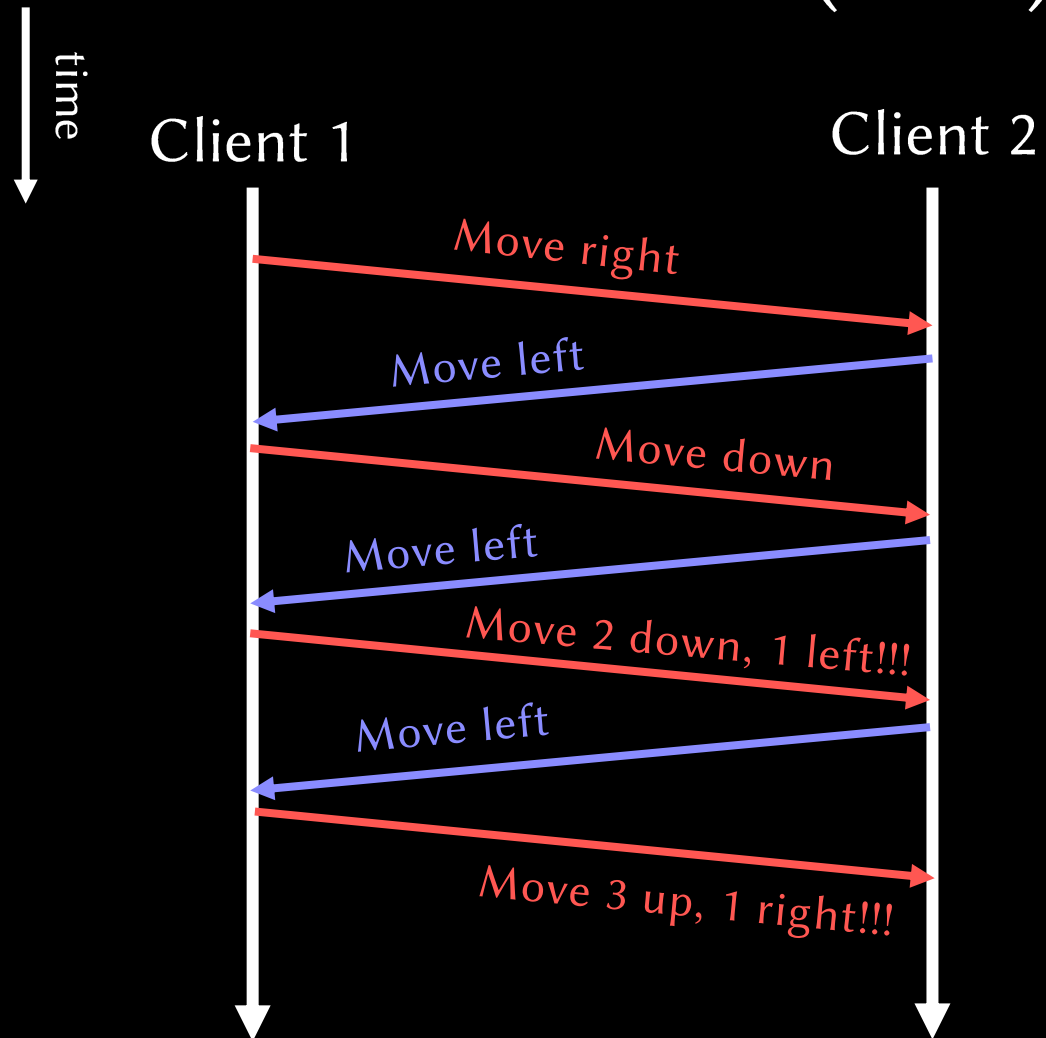


2025

Peer-to-Peer (P2P)



Peer-to-Peer (P2P)



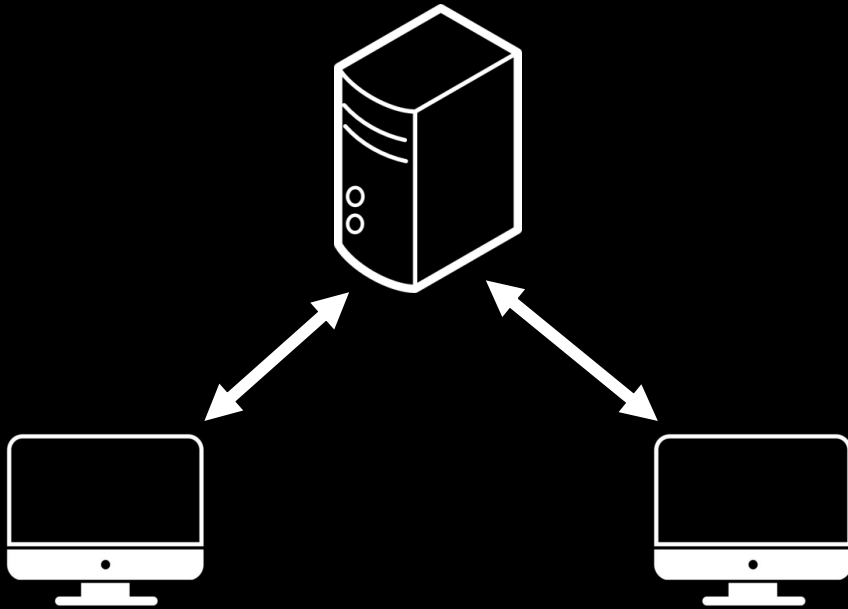
Problem:

Players cheat.

Always assume the worst.

How to prevent cheating?

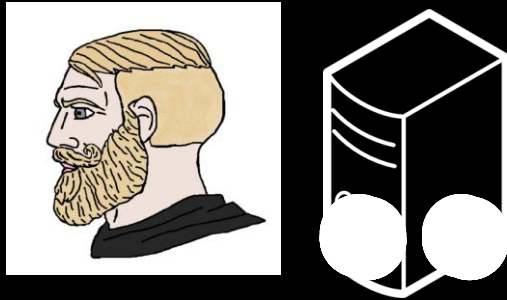
Solution: Use an authoritative server.



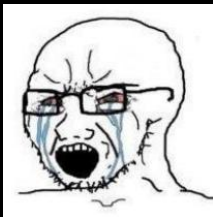
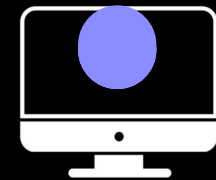
Client-server architecture

- Server holds all of the game state.
- Clients send inputs to server.
- Server verifies inputs, updates game state.
- Server constantly sends updates to all clients every “tick” (at least 20 times per second).

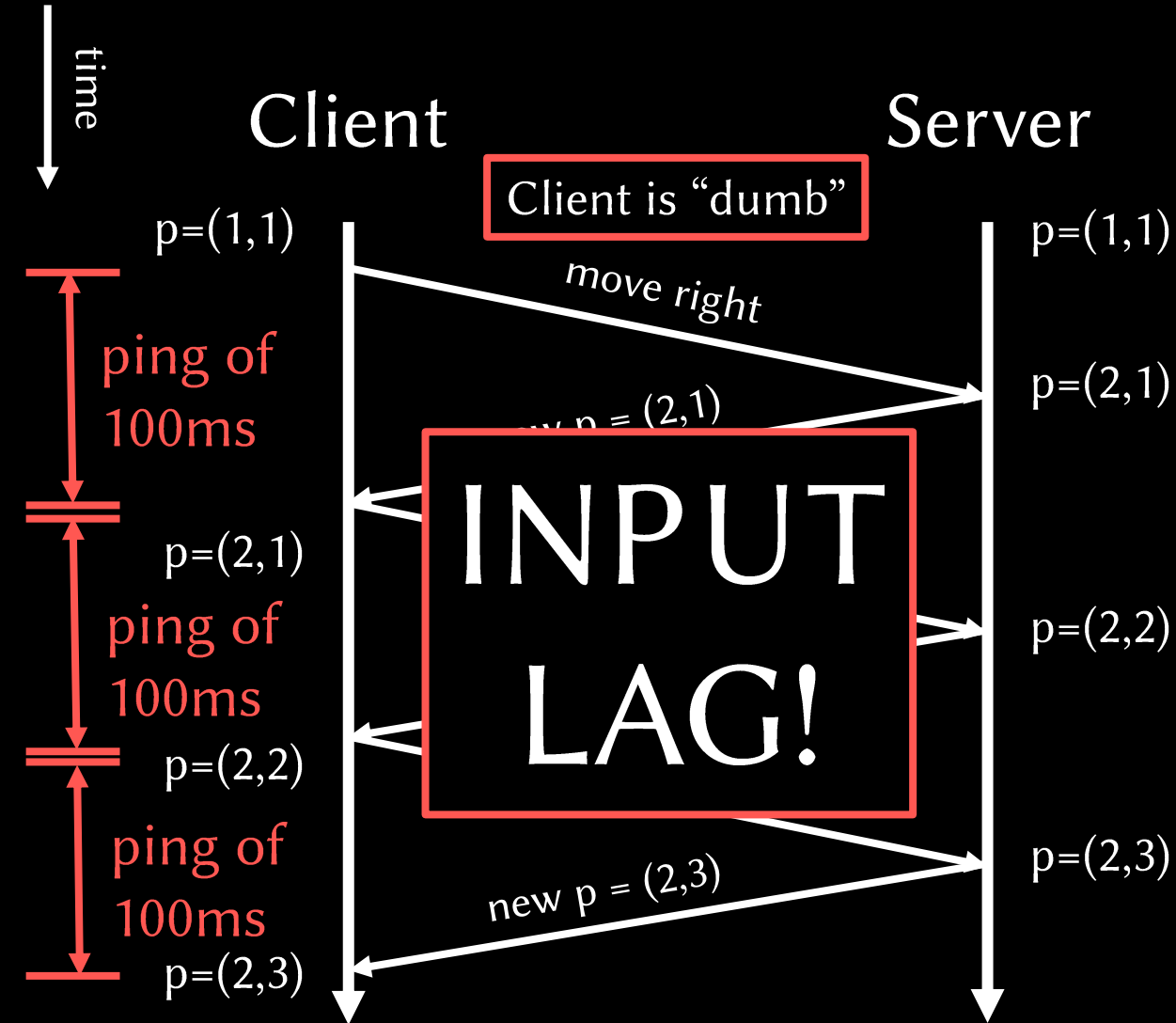
Authoritative server



- The server is constantly blasting the clients with updates.
- Clients sometimes send the server requests.
- Clients must always listen to the server.



Client-server



Client view

	1	2	3	4
1				
2				
3				
4				

Problem:

Input lag

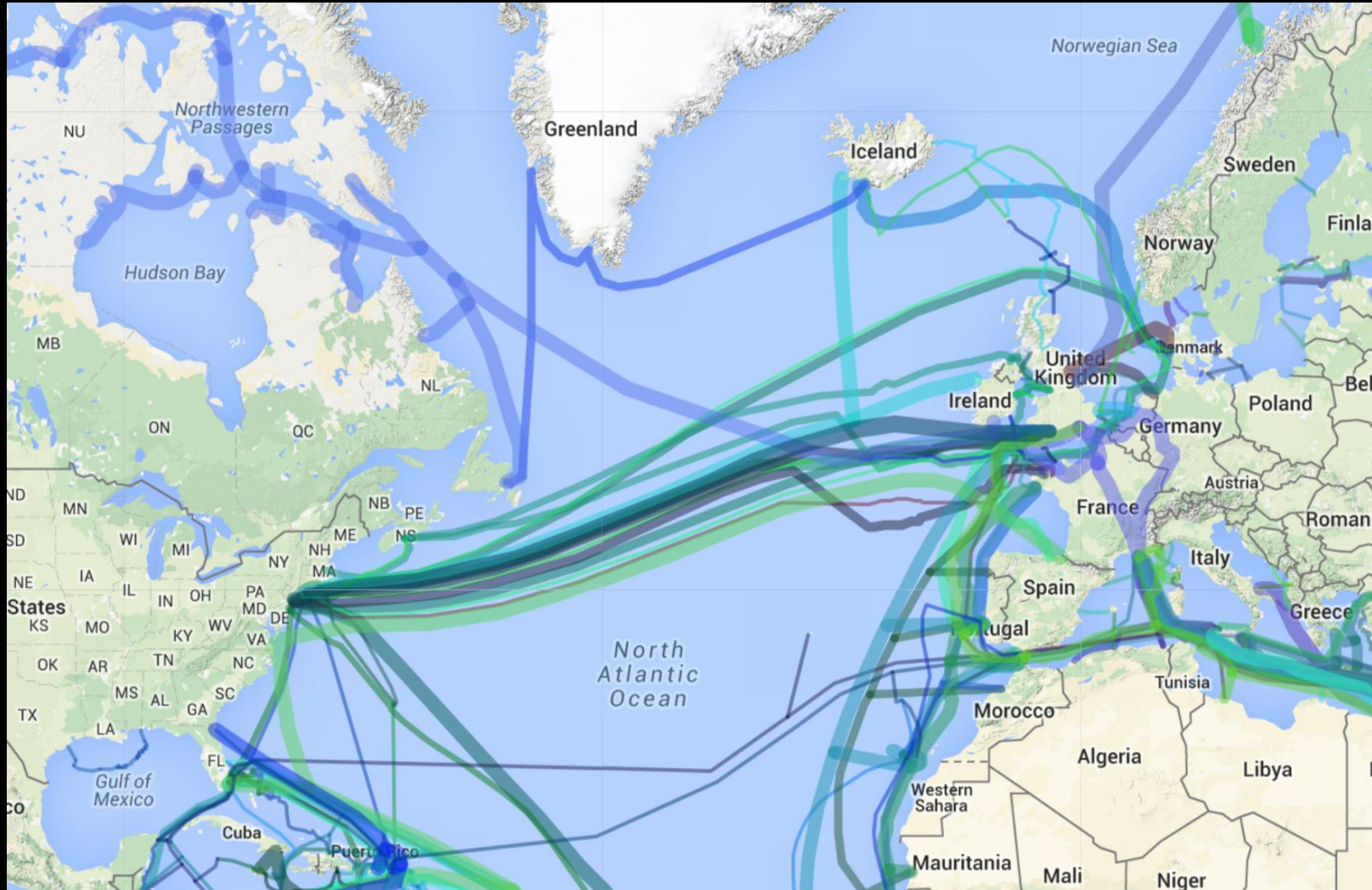
makes games unplayable.

Detour

Why does lag occur?

Why does lag occur?

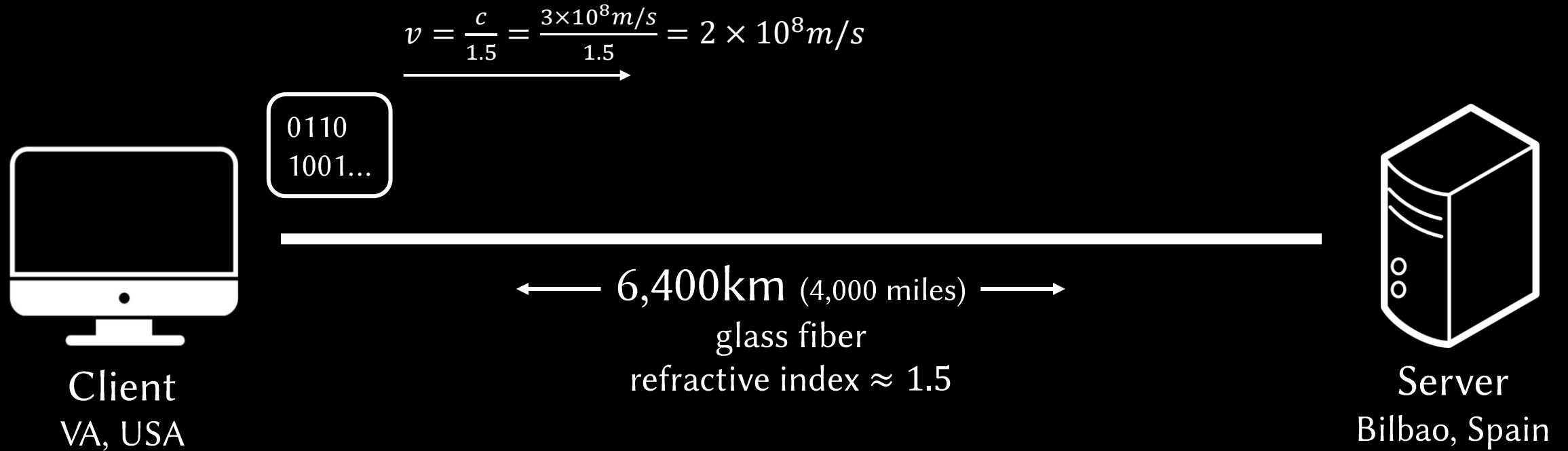
Reason 1: Information has to travel far.



Transatlantic underwater cables

Why does lag occur?

Reason 1: Information has to travel far.

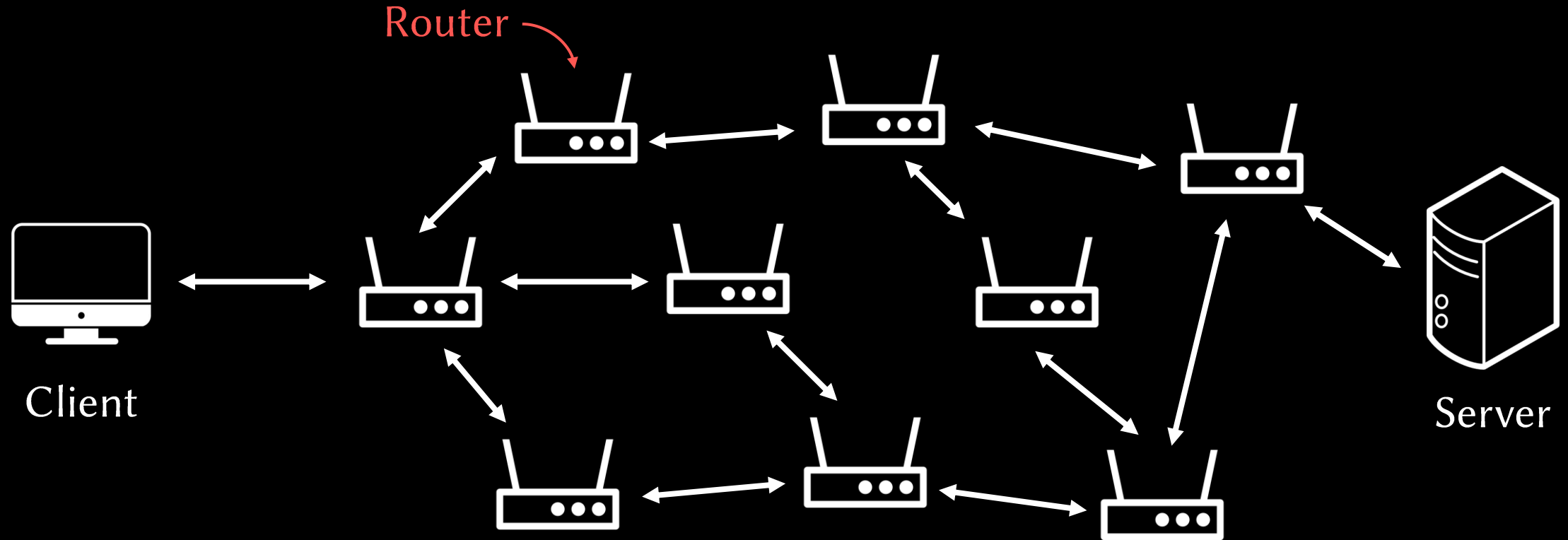


Packet would take

$$t = \frac{6400 \times 10^3 m}{2 \times 10^8 m/s} = 32 \text{ ms!}$$

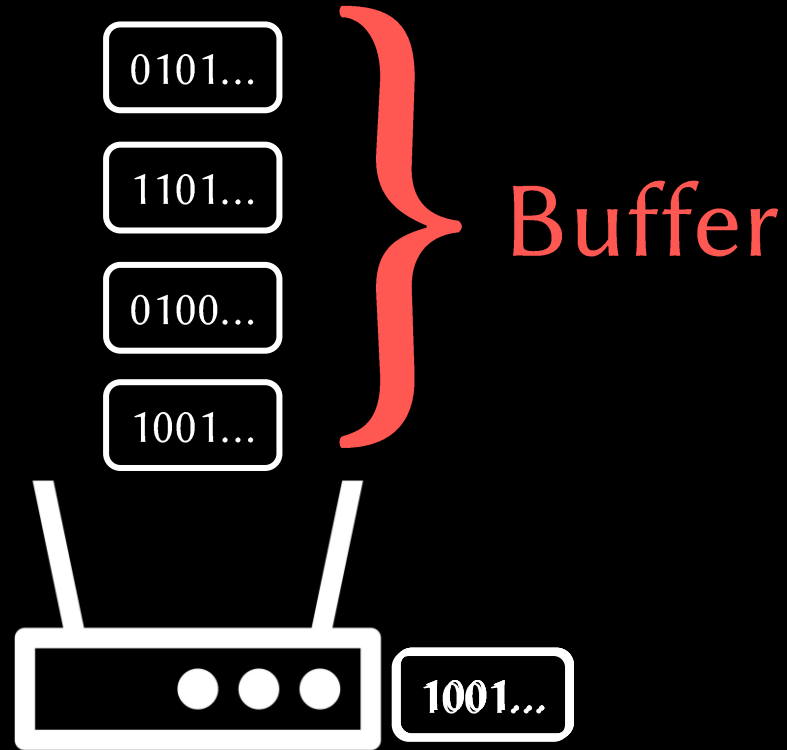
Why does lag occur?

Reason 2: The internet is complicated.



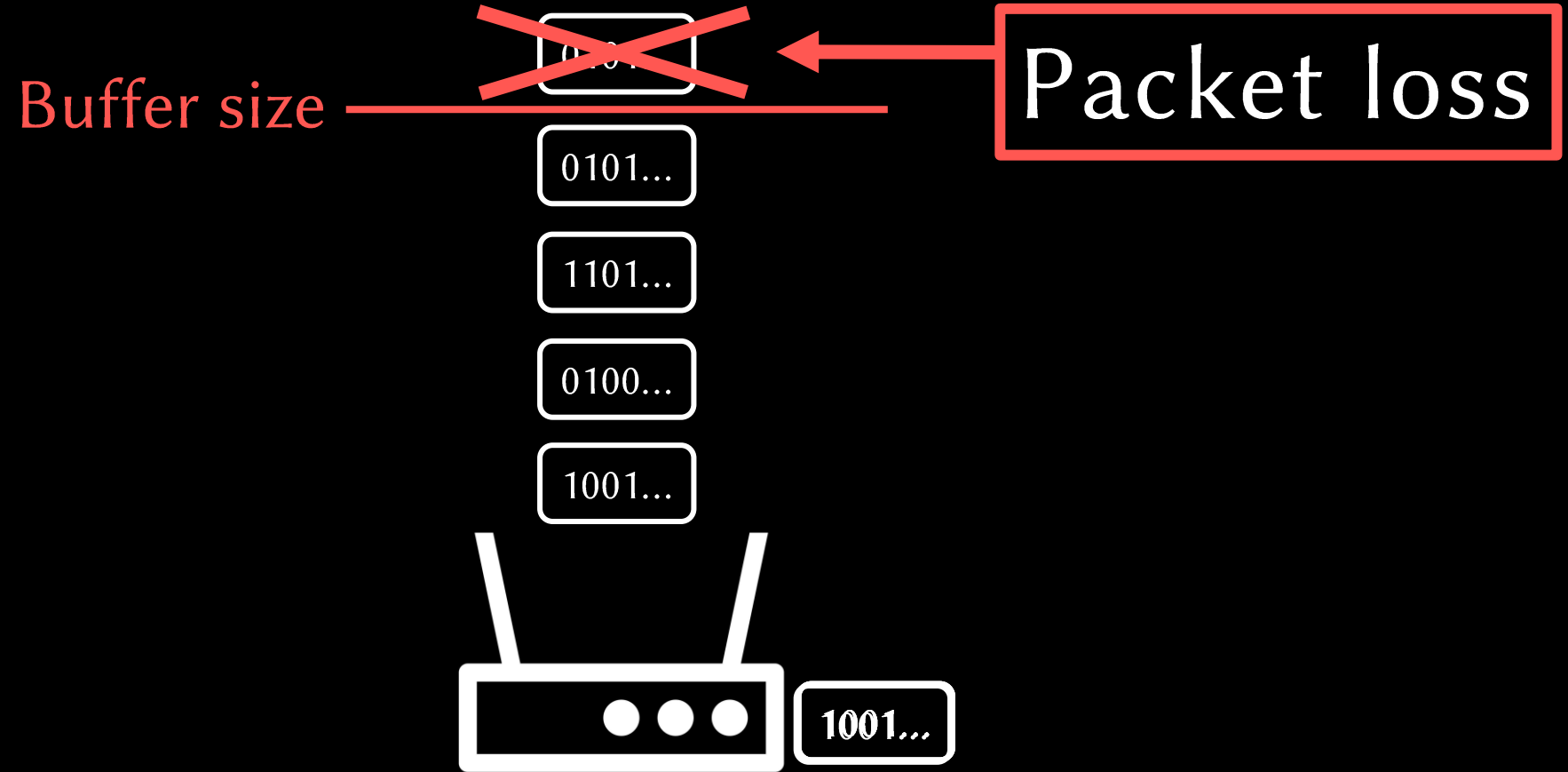
Why does lag occur?

Reason 3: Routers are slow.



Why does lag occur?

Reason 4: Router buffers are finite.



Why lag occurs: Summary

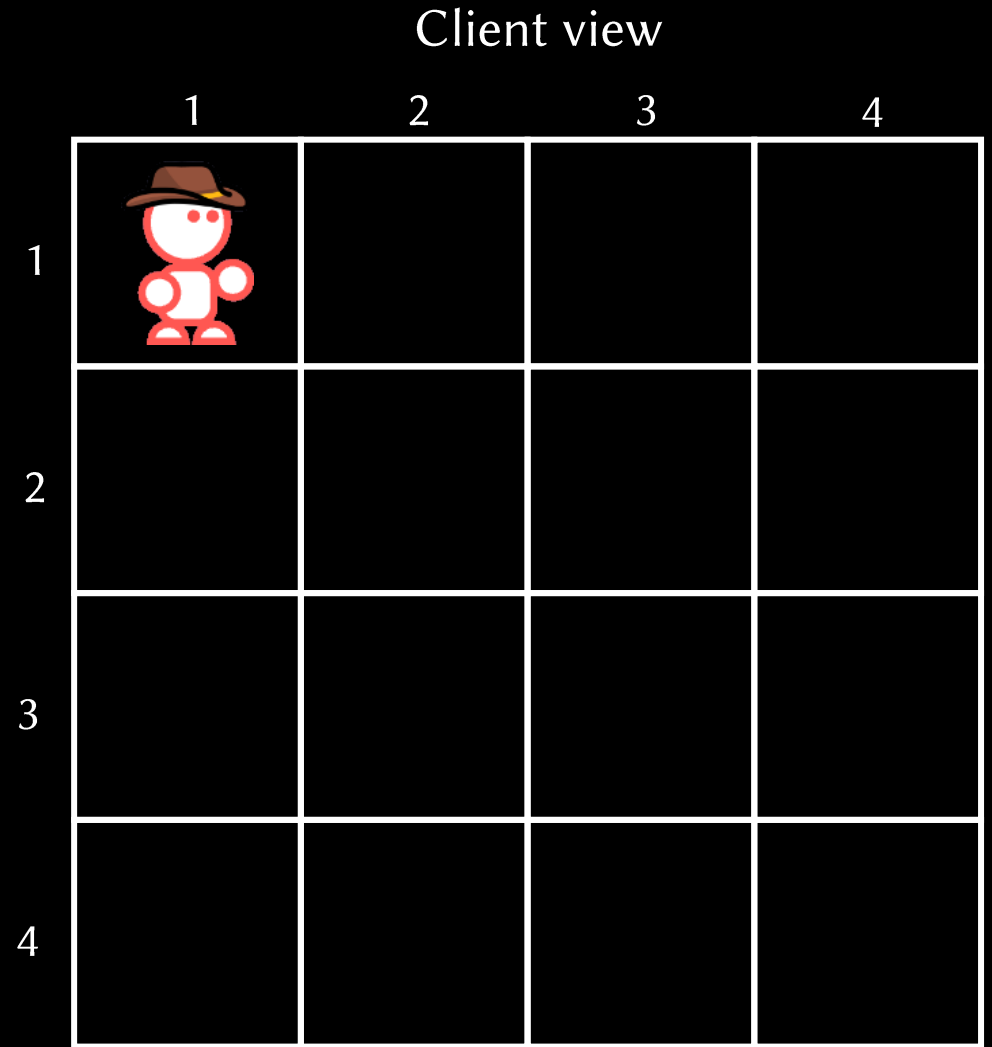
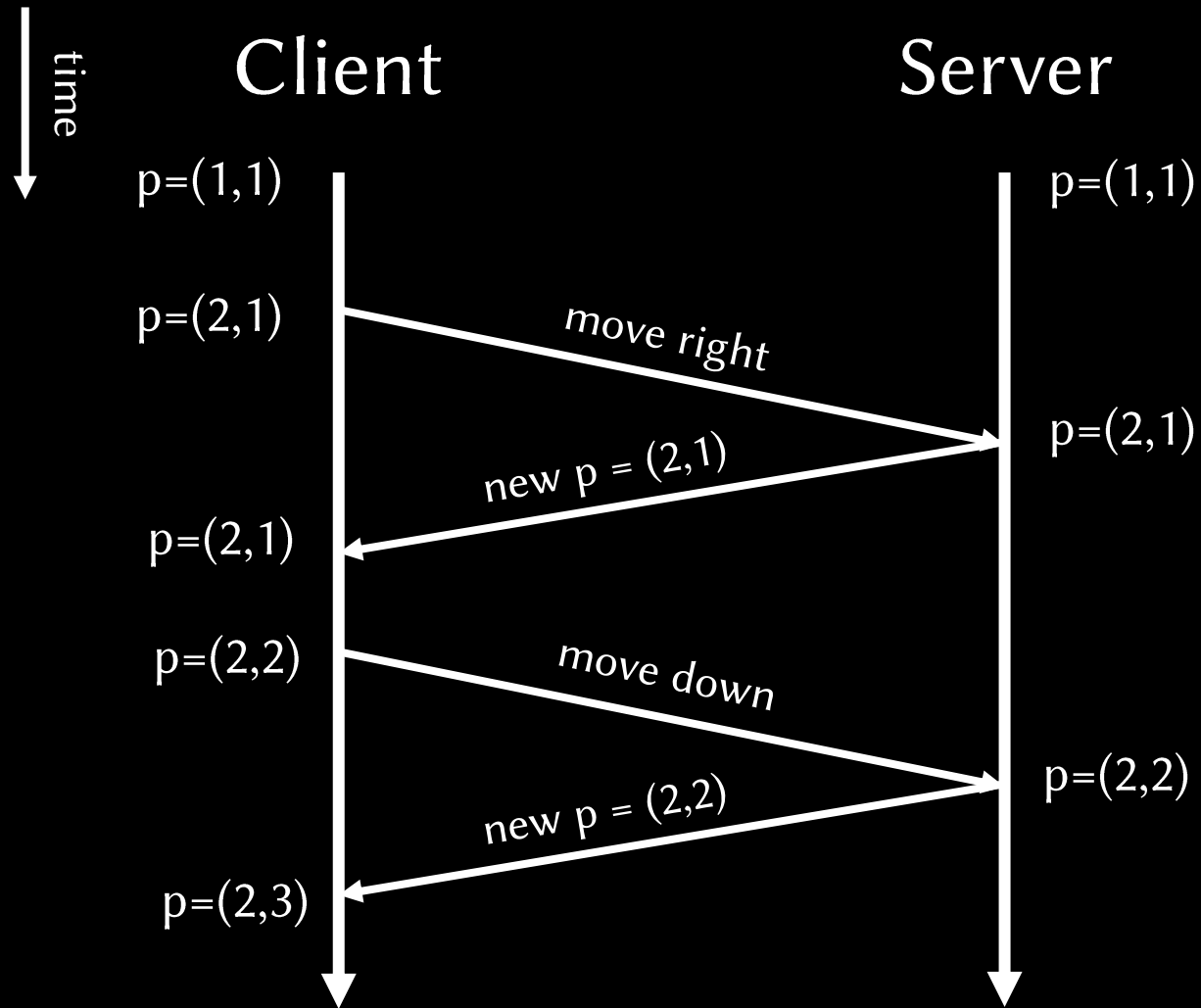
1. Information has to travel far.
2. Information travels a complicated route through many routers.
3. Routers are slow, especially during congestion.
4. Routers can drop packets, (sometimes) needing them to be resent.

Let's go back to input lag.

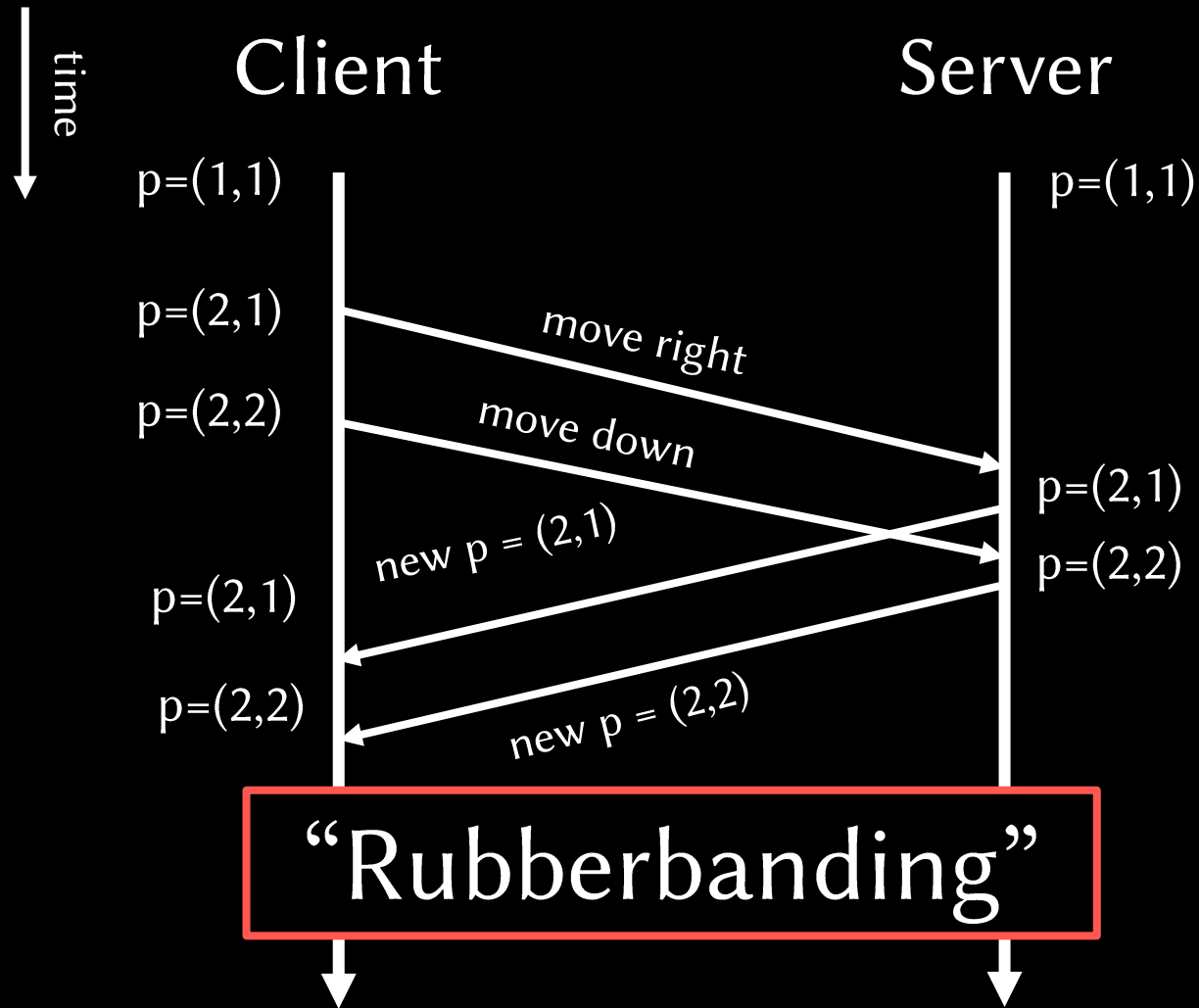
How to reduce the effect of lag for the client?

Solution: Client predicts its state.

Client-side prediction



Client-side prediction goes wrong!

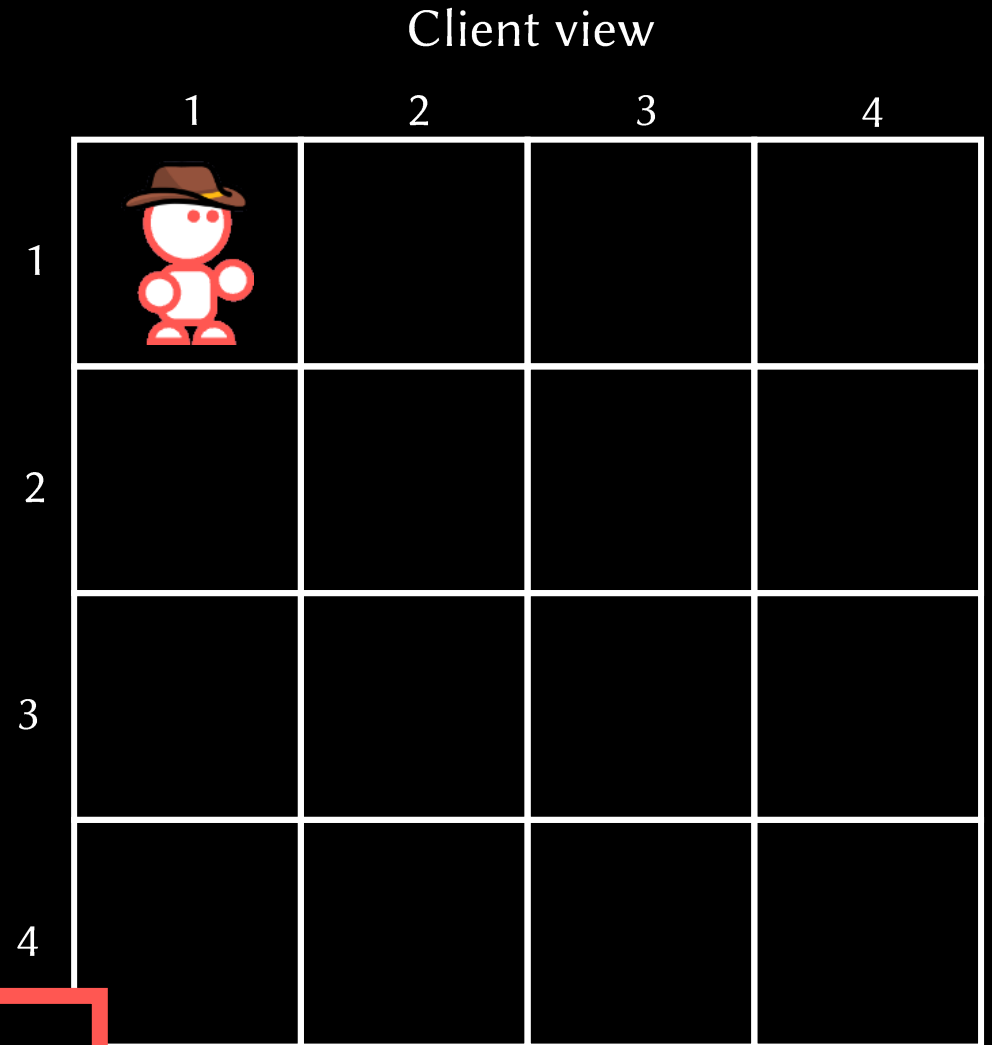
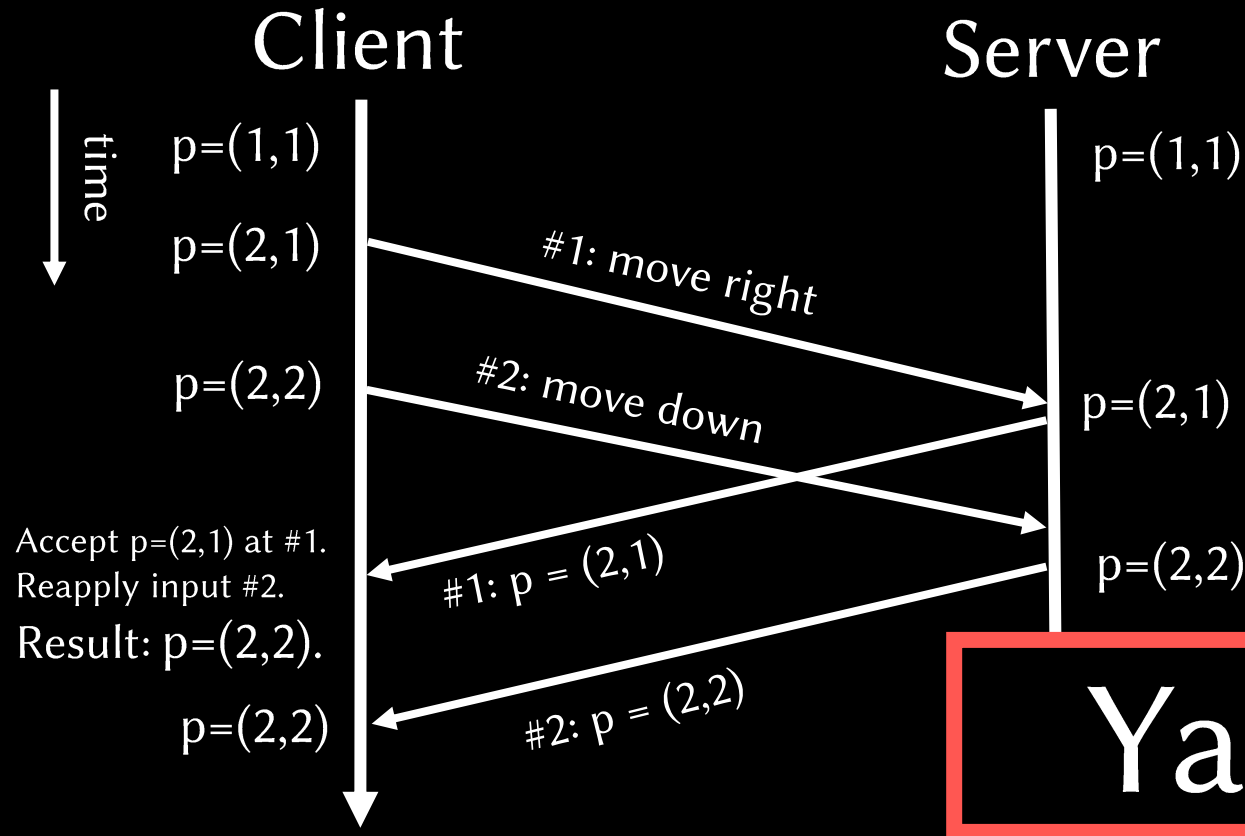


Client view

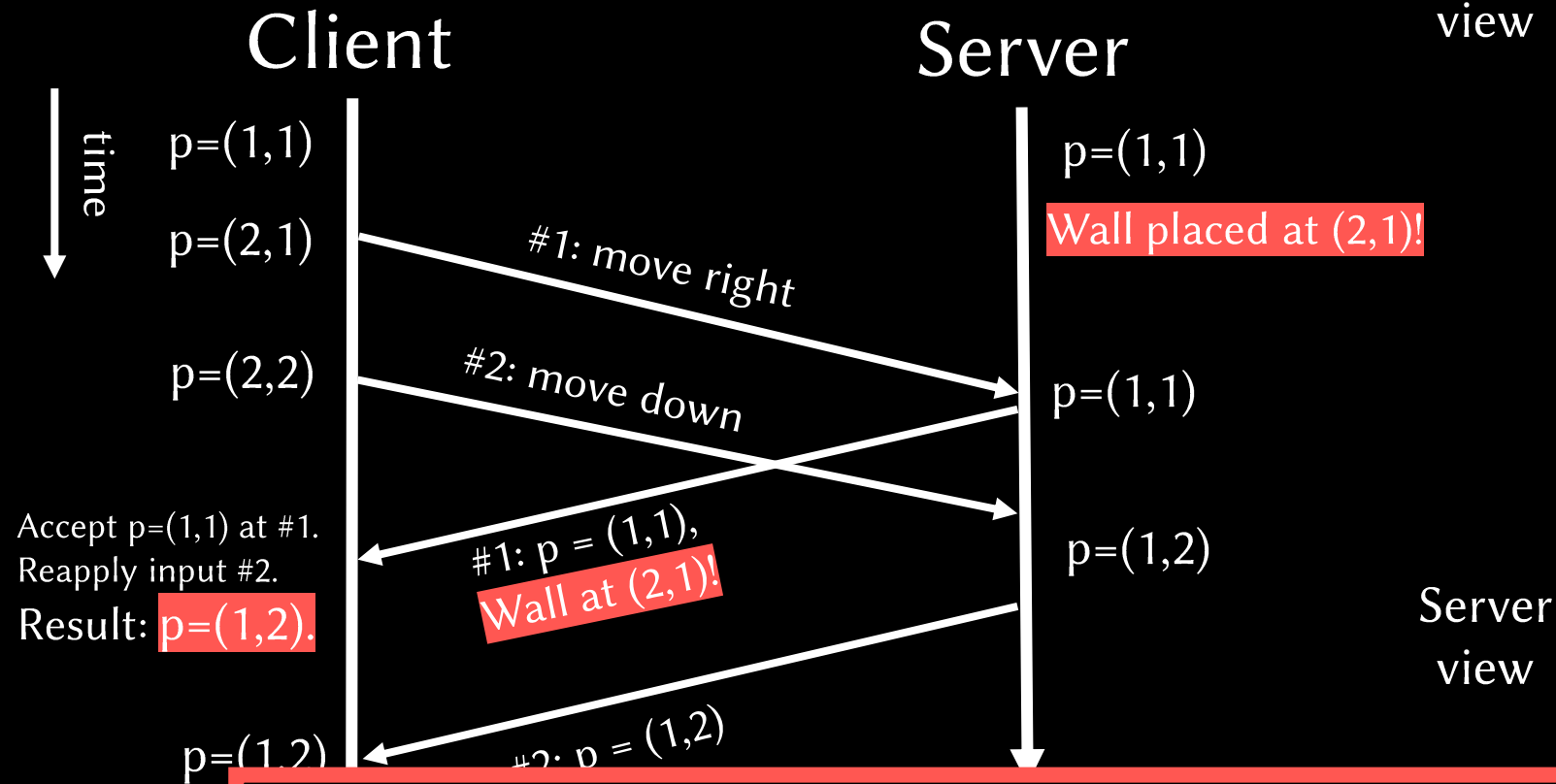
	1	2	3	4
1				
2				
3				
4				

Server reconciliation

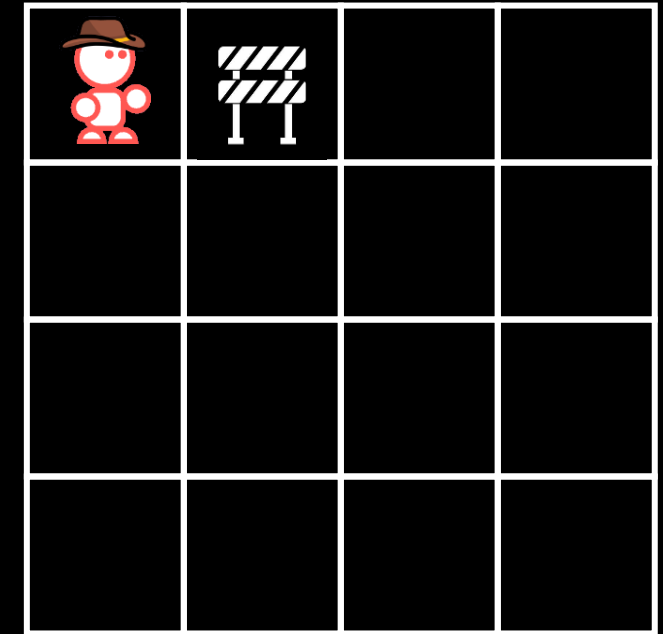
- Client remembers what inputs it made.
- When it receives a reply from the server, it accepts it and reapplies all unprocessed inputs.



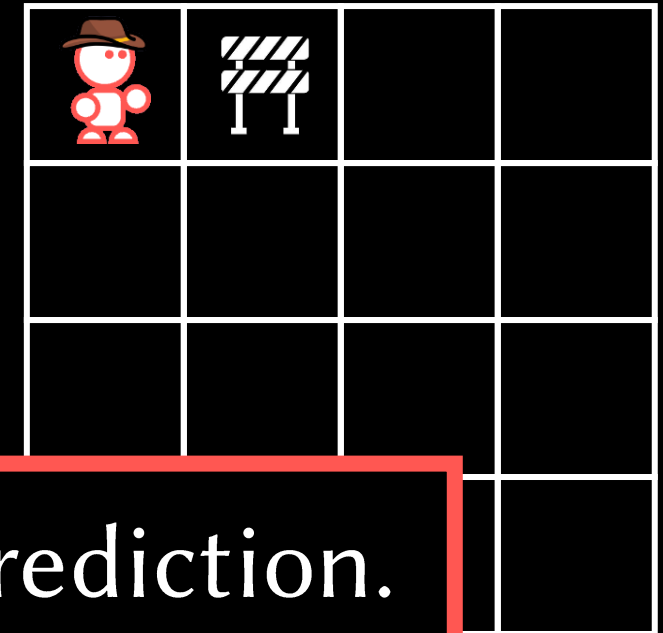
Server reconciliation



Client
view



Server
view



Not ideal, but better than no prediction.

Demo:

Prediction + reconciliation in action

Issues with client prediction

- The client plays the game in the future and sees other players in the past.
- This results in peeker's advantage:
 - Higher ping players get an advantage over lower ping players.

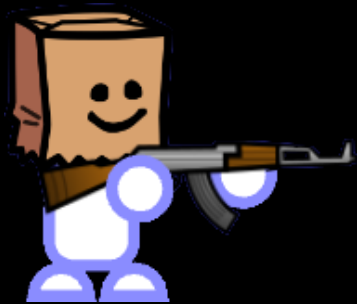
Peeker's advantage



Another issue with prediction: Aiming

- Since the client sees other players in the past, they'll aim at players who aren't there anymore!

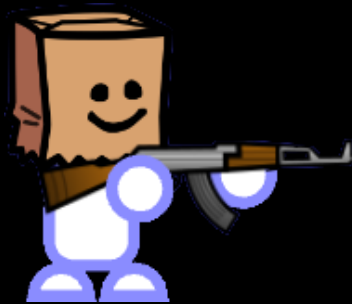
What client sees:



Another issue with prediction: Aiming

- Since the client sees other players in the past, they'll aim at players who aren't there anymore!

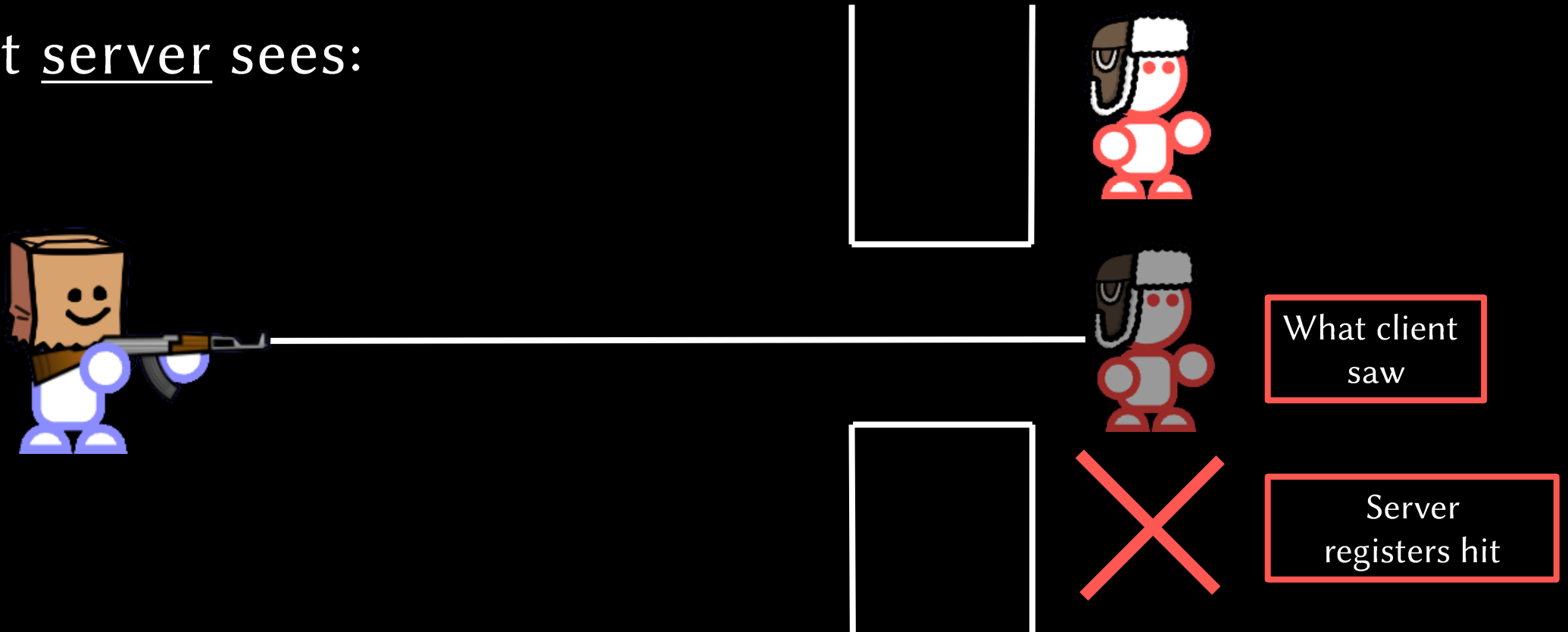
What server sees:



Solution: Server-side rewind

- The server can determine the client's ping (how?), so it can figure out what the client saw when they fired.
- The server rolls back the game state to see if the client landed a hit.

What server sees:



Solution: Server-side rewind

- The server knows the client's ping, so it can figure out what the client saw when they fired.
- The server rolls back the game state to see if the client landed a hit.

What other player sees:



Can be unfair for player getting shot!



Hopefully, you get the point...

For every solution, there's always some case where things suck for someone.

That's the beauty of game networking 😊

Conclusion

- Game networking is complicated!
- Lag is inevitable: info must travel far through unreliable routers.
- To prevent cheating, use an authoritative server.
- To hide input lag, use client-side prediction with reconciliation.
- Use server-side rewind to make shooting fair for players with high ping.

Keep blaming the game developers for your missed shots!



Thank you!

